

OS Lab Exam Complete Study Guide

1. UNIX System Calls

Problem 1: Process Creation using fork()

Question: Write a program to create a child process and display PIDs of parent and child.

```
c

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child Process: PID = %d, Parent PID = %d\n", getpid(), getppid());
    } else if (pid > 0) {
        // Parent process
        printf("Parent Process: PID = %d, Child PID = %d\n", getpid(), pid);
    } else {
        printf("Fork failed!\n");
    }

    return 0;
}
```

Problem 2: Using exec() family

Question: Write a program to execute another program using exec().

```
c
```

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process - execute ls command
        execl("/bin/ls", "ls", "-l", NULL);
        printf("exec failed!\n"); // This line executes only if exec fails
    } else if (pid > 0) {
        // Parent process
        wait(NULL); // Wait for child to complete
        printf("Child process completed\n");
    }

    return 0;
}

```

Problem 3: File Operations with System Calls

Question: Write a program to read file information using stat().

```

c

#include <stdio.h>
#include <sys/stat.h>
#include <time.h>

int main() {
    struct stat fileStat;
    char filename[] = "test.txt";

    if (stat(filename, &fileStat) == 0) {
        printf("File: %s\n", filename);
        printf("Size: %ld bytes\n", fileStat.st_size);
        printf("Permissions: %o\n", fileStat.st_mode & 0777);
        printf("Last modified: %s", ctime(&fileStat.st_mtime));
    } else {
        perror("stat");
    }

    return 0;
}

```

Problem 4: Directory Operations

Question: Write a program to list directory contents using opendir/readdir.

```
c

#include <stdio.h>
#include <dirent.h>
#include <stdlib.h>

int main() {
    DIR *dir;
    struct dirent *entry;

    dir = opendir(".");
    if (dir == NULL) {
        perror("opendir");
        exit(1);
    }

    printf("Directory contents:\n");
    while ((entry = readdir(dir)) != NULL) {
        printf("%s\n", entry->d_name);
    }

    closedir(dir);
    return 0;
}
```

2. CPU Scheduling Algorithms

Problem 5: FCFS Scheduling

Question: Implement First Come First Serve scheduling algorithm.

```
c
```

```
#include <stdio.h>
```

```
struct Process {  
    int pid;  
    int arrival_time;  
    int burst_time;  
    int completion_time;  
    int turnaround_time;  
    int waiting_time;  
};
```

```
void fcfs(struct Process proc[], int n) {  
    int current_time = 0;
```

```
    // Sort by arrival time (assuming already sorted)
```

```
    for (int i = 0; i < n; i++) {  
        if (current_time < proc[i].arrival_time) {  
            current_time = proc[i].arrival_time;  
        }  
  
        proc[i].completion_time = current_time + proc[i].burst_time;  
        proc[i].turnaround_time = proc[i].completion_time - proc[i].arrival_time;  
        proc[i].waiting_time = proc[i].turnaround_time - proc[i].burst_time;  
  
        current_time = proc[i].completion_time;  
    }  
}
```

```
int main() {  
    int n = 3;  
    struct Process proc[] = {{1, 0, 5}, {2, 1, 3}, {3, 2, 8}};  
  
    fcfs(proc, n);  
  
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");  
    for (int i = 0; i < n; i++) {  
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",  
            proc[i].pid, proc[i].arrival_time, proc[i].burst_time,  
            proc[i].completion_time, proc[i].turnaround_time, proc[i].waiting_time);  
    }  
  
    return 0;  
}
```

Problem 6: SJF Scheduling

Question: Implement Shortest Job First (Non-preemptive) scheduling.

c

```
#include <stdio.h>
#include <limits.h>
```

```
struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int completion_time;
    int turnaround_time;
    int waiting_time;
    int completed;
};
```

```
void sjf(struct Process proc[], int n) {
    int current_time = 0;
    int completed = 0;
```

```
    while (completed < n) {
        int idx = -1;
        int min_burst = INT_MAX;
```

```
        // Find process with minimum burst time that has arrived
```

```
        for (int i = 0; i < n; i++) {
            if (!proc[i].completed && proc[i].arrival_time <= current_time &&
                proc[i].burst_time < min_burst) {
                min_burst = proc[i].burst_time;
                idx = i;
            }
        }
```

```
        if (idx == -1) {
            current_time++;
            continue;
        }
```

```
        proc[idx].completion_time = current_time + proc[idx].burst_time;
        proc[idx].turnaround_time = proc[idx].completion_time - proc[idx].arrival_time;
        proc[idx].waiting_time = proc[idx].turnaround_time - proc[idx].burst_time;
        proc[idx].completed = 1;
```

```
        current_time = proc[idx].completion_time;
        completed++;
```

```
    }
}
```

```
int main() {
```

```

int n = 4;
struct Process proc[] = {{1, 0, 6, 0, 0, 0, 0}, {2, 2, 8, 0, 0, 0, 0},
                        {3, 1, 7, 0, 0, 0, 0}, {4, 3, 3, 0, 0, 0, 0}};

sjf(proc, n);

printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        proc[i].pid, proc[i].arrival_time, proc[i].burst_time,
        proc[i].completion_time, proc[i].turnaround_time, proc[i].waiting_time);
}

return 0;
}

```

Problem 7: Round Robin Scheduling

Question: Implement Round Robin scheduling with time quantum.

c

```
#include <stdio.h>
```

```
struct Process {  
    int pid;  
    int arrival_time;  
    int burst_time;  
    int remaining_time;  
    int completion_time;  
    int turnaround_time;  
    int waiting_time;  
};
```

```
void roundRobin(struct Process proc[], int n, int quantum) {  
    int current_time = 0;  
    int completed = 0;
```

```
    // Initialize remaining time
```

```
    for (int i = 0; i < n; i++) {  
        proc[i].remaining_time = proc[i].burst_time;  
    }
```

```
    while (completed < n) {  
        int progress = 0;
```

```
        for (int i = 0; i < n; i++) {  
            if (proc[i].remaining_time > 0 && proc[i].arrival_time <= current_time) {  
                progress = 1;
```

```
                if (proc[i].remaining_time > quantum) {  
                    current_time += quantum;  
                    proc[i].remaining_time -= quantum;  
                } else {  
                    current_time += proc[i].remaining_time;  
                    proc[i].completion_time = current_time;  
                    proc[i].turnaround_time = proc[i].completion_time - proc[i].arrival_time;  
                    proc[i].waiting_time = proc[i].turnaround_time - proc[i].burst_time;  
                    proc[i].remaining_time = 0;  
                    completed++;  
                }  
            }  
        }  
    }
```

```
    if (!progress) current_time++;  
}
```



```

int main() {
    int n = 3;
    int quantum = 2;
    struct Process proc[] = {{1, 0, 5}, {2, 1, 4}, {3, 2, 2}};

    roundRobin(proc, n, quantum);

    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            proc[i].pid, proc[i].arrival_time, proc[i].burst_time,
            proc[i].completion_time, proc[i].turnaround_time, proc[i].waiting_time);
    }

    return 0;
}

```

3. Thread Programming

Problem 8: Basic Thread Creation

Question: Write a program to create threads using pthread.

c

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void* threadFunction(void* arg) {
    int thread_id = *(int*)arg;
    printf("Thread %d is running\n", thread_id);
    sleep(1);
    printf("Thread %d completed\n", thread_id);
    return NULL;
}

int main() {
    pthread_t threads[3];
    int thread_ids[3] = {1, 2, 3};

    // Create threads
    for (int i = 0; i < 3; i++) {
        pthread_create(&threads[i], NULL, threadFunction, &thread_ids[i]);
    }

    // Wait for all threads to complete
    for (int i = 0; i < 3; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("All threads completed\n");
    return 0;
}

```

4. Synchronization - Producer-Consumer Problem

Problem 9: Producer-Consumer using Semaphores

Question: Implement producer-consumer problem using semaphores.

c

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full, mutex;

void* producer(void* arg) {
    int item = 1;
    while (1) {
        sem_wait(&empty); // Wait for empty slot
        sem_wait(&mutex); // Enter critical section

        buffer[in] = item;
        printf("Producer produced item %d at position %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        item++;

        sem_post(&mutex); // Exit critical section
        sem_post(&full); // Signal full slot
        sleep(1);
    }
}

void* consumer(void* arg) {
    int item;
    while (1) {
        sem_wait(&full); // Wait for full slot
        sem_wait(&mutex); // Enter critical section

        item = buffer[out];
        printf("Consumer consumed item %d from position %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        sem_post(&mutex); // Exit critical section
        sem_post(&empty); // Signal empty slot
        sleep(2);
    }
}

int main() {
```

```
pthread_t prod_thread, cons_thread;

// Initialize semaphores
sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&full, 0, 0);
sem_init(&mutex, 0, 1);

// Create threads
pthread_create(&prod_thread, NULL, producer, NULL);
pthread_create(&cons_thread, NULL, consumer, NULL);

// Wait for threads
pthread_join(prod_thread, NULL);
pthread_join(cons_thread, NULL);

return 0;
}
```

5. Banker's Algorithm

Problem 10: Banker's Algorithm for Deadlock Avoidance

Question: Implement Banker's algorithm to check safe state.

c

```

#include <stdio.h>
#include <stdbool.h>

#define P 5 // Number of processes
#define R 3 // Number of resources

bool isSafe(int processes[], int avail[], int max[][R], int allot[][R]) {
    int need[P][R];

    // Calculate need matrix
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            need[i][j] = max[i][j] - allot[i][j];
        }
    }

    bool finish[P] = {false};
    int safeSeq[P];
    int work[R];

    // Initialize work with available resources
    for (int i = 0; i < R; i++) {
        work[i] = avail[i];
    }

    int count = 0;
    while (count < P) {
        bool found = false;

        for (int p = 0; p < P; p++) {
            if (!finish[p]) {
                int j;
                for (j = 0; j < R; j++) {
                    if (need[p][j] > work[j]) break;
                }

                if (j == R) {
                    // Add allocated resources back to work
                    for (int k = 0; k < R; k++) {
                        work[k] += allot[p][k];
                    }

                    safeSeq[count++] = p;
                    finish[p] = true;
                    found = true;
                }
            }
        }
    }
}

```

```

    }
}

if (!found) {
    printf("System is not in safe state\n");
    return false;
}

printf("System is in safe state.\nSafe sequence: ");
for (int i = 0; i < P; i++) {
    printf("%d ", safeSeq[i]);
}
printf("\n");

return true;
}

int main() {
    int processes[] = {0, 1, 2, 3, 4};
    int avail[] = {3, 3, 2};

    int max[][R] = {{7, 5, 3},
                    {3, 2, 2},
                    {9, 0, 2},
                    {2, 2, 2},
                    {4, 3, 3}};

    int allot[][R] = {{0, 1, 0},
                    {2, 0, 0},
                    {3, 0, 2},
                    {2, 1, 1},
                    {0, 0, 2}};

    isSafe(processes, avail, max, allot);
    return 0;
}

```

6. Memory Management - Paging

Problem 11: Page Replacement - FIFO Algorithm

Question: Implement FIFO page replacement algorithm.

c

```
#include <stdio.h>
#include <stdbool.h>

int findPage(int frames[], int n, int page) {
    for (int i = 0; i < n; i++) {
        if (frames[i] == page) return i;
    }
    return -1;
}

void fifoPageReplacement(int pages[], int n, int capacity) {
    int frames[capacity];
    int front = 0;
    int page_faults = 0;

    // Initialize frames
    for (int i = 0; i < capacity; i++) {
        frames[i] = -1;
    }

    printf("Page Reference String: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", pages[i]);
    }
    printf("\n\n");

    for (int i = 0; i < n; i++) {
        int page = pages[i];

        if (findPage(frames, capacity, page) == -1) {
            frames[front] = page;
            front = (front + 1) % capacity;
            page_faults++;

            printf("Page %d -> Page Fault | Frames: ", page);
        } else {
            printf("Page %d -> Hit | Frames: ", page);
        }

        for (int j = 0; j < capacity; j++) {
            if (frames[j] == -1) printf("- ");
            else printf("%d ", frames[j]);
        }
        printf("\n");
    }
}
```

```
printf("\nTotal Page Faults: %d\n", page_faults);
}

int main() {
    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};
    int n = sizeof(pages) / sizeof(pages[0]);
    int capacity = 3;

    fifoPageReplacement(pages, n, capacity);
    return 0;
}
```

7. Bash Scripting

Problem 12: Basic Bash Script Examples

bash


```
#!/bin/bash
```

```
# Script 1: File operations and conditions
```

```
echo "Enter filename:"
```

```
read filename
```

```
if [ -f "$filename" ]; then
```

```
    echo "File exists"
```

```
    echo "File size: $(wc -c < $filename) bytes"
```

```
    echo "File permissions: $(stat -c %A $filename)"
```

```
else
```

```
    echo "File does not exist"
```

```
fi
```

```
# Script 2: Loop and arithmetic
```

```
echo "Enter a number:"
```

```
read n
```

```
echo "Multiplication table of $n:"
```

```
for i in {1..10}; do
```

```
    result=$((n * i))
```

```
    echo "$n x $i = $result"
```

```
done
```

```
# Script 3: Process information
```

```
echo "Current processes by current user:"
```

```
ps -u $USER
```

```
echo "System information:"
```

```
echo "Current date: $(date)"
```

```
echo "Current directory: $(pwd)"
```

```
echo "Current user: $USER"
```

```
echo "Home directory: $HOME"
```

VIVA QUESTIONS AND ANSWERS

1. Process Management

Q: What is the difference between fork() and exec()? A: fork() creates a copy of the current process (child process), while exec() replaces the current process image with a new program. fork() returns PID of child to parent and 0 to child, exec() doesn't return if successful.

Q: What happens when a parent process terminates before child? A: The child becomes an orphan process and is adopted by init process (PID 1).

Q: What is a zombie process? A: A process that has completed execution but still has an entry in process table. Occurs when parent hasn't called wait() to read child's exit status.

2. CPU Scheduling

Q: Why is SJF optimal for average waiting time? A: SJF minimizes average waiting time because shorter jobs complete first, reducing the waiting time of subsequent processes.

Q: What is starvation in scheduling? A: When a process waits indefinitely because other processes continuously get CPU time. Common in priority scheduling.

Q: How does Round Robin handle starvation? A: RR prevents starvation by giving each process equal time slice in circular manner.

3. Synchronization

Q: What is the difference between mutex and semaphore? A: Mutex is binary (0 or 1) used for mutual exclusion. Semaphore can have any value and is used for resource counting and synchronization.

Q: What is critical section? A: A code segment where shared resources are accessed. Only one process should execute in critical section at a time.

Q: What conditions must be satisfied for critical section solution? A: Mutual exclusion, progress, and bounded waiting.

4. Deadlock

Q: What are the four conditions for deadlock? A: 1) Mutual exclusion 2) Hold and wait 3) No preemption 4) Circular wait

Q: How does Banker's algorithm prevent deadlock? A: It checks if resource allocation leads to safe state. If allocation results in unsafe state, request is denied.

5. Memory Management

Q: What is the difference between internal and external fragmentation? A: Internal fragmentation: unused space within allocated memory block. External fragmentation: free memory scattered in small blocks.

Q: How does paging solve fragmentation? A: Paging eliminates external fragmentation by dividing memory into fixed-size frames and processes into pages.

6. File Systems

Q: What is the difference between absolute and relative path? A: Absolute path starts from root directory (/), relative path starts from current directory.

Q: What does stat() system call return? A: File metadata including size, permissions, timestamps, inode number, etc.

7. Threads

Q: What is the difference between process and thread? A: Processes have separate memory space, threads share memory space within a process. Thread creation is faster and requires less overhead.

Q: What are the advantages of multithreading? A: Better resource utilization, faster execution, responsiveness, and easier communication between threads.

8. Shell Programming

Q: What is the difference between \$? and \$#? A: \$? contains exit status of last command, \$# contains number of command-line arguments.

Q: How do you make a script executable? A: Use `chmod +x scriptname.sh` or `chmod 755 scriptname.sh`

IMPORTANT COMPILATION COMMANDS

```
bash

# For programs using pthread
gcc -pthread program.c -o program

# For programs using semaphores
gcc -pthread program.c -o program

# For programs using math library
gcc -lm program.c -o program

# General compilation
gcc program.c -o program
```

QUICK REFERENCE

System Calls

- `fork()` - Create child process
- `exec()` - Execute new program
- `wait()` - Wait for child process
- `getpid()` - Get process ID
- `stat()` - Get file information
- `opendir()`, `readdir()` - Directory operations

Scheduling Metrics

- **Turnaround Time** = Completion Time - Arrival Time
- **Waiting Time** = Turnaround Time - Burst Time
- **Response Time** = First CPU allocation - Arrival Time

Semaphore Operations

- `sem_init()` - Initialize semaphore
- `sem_wait()` - Decrement (P operation)
- `sem_post()` - Increment (V operation)

8. Advanced Inter-Process Communication

Problem 13: Shared Memory Implementation

Question: Implement shared memory communication between processes.

c

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/types.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <unistd.h>

#define SHM_SIZE 1024

// Writer Process
void writer_process() {
    key_t key = ftok("shmfile", 65);
    int shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);

    char *str = (char*) shmat(shmid, (void*)0, 0);

    printf("Writer: Enter data to write to shared memory: ");
    fgets(str, SHM_SIZE, stdin);

    printf("Writer: Data written to shared memory: %s\n", str);
    shmdt(str);
}

// Reader Process
void reader_process() {
    key_t key = ftok("shmfile", 65);
    int shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);

    char *str = (char*) shmat(shmid, (void*)0, 0);

    printf("Reader: Data read from shared memory: %s\n", str);

    shmdt(str);
    shmctl(shmid, IPC_RMID, NULL); // Remove shared memory
}

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process - reader
        sleep(1); // Ensure writer writes first
        reader_process();
    } else {
```

```
// Parent process - writer
writer_process();
wait(NULL);
}

return 0;
}
```

Problem 14: Message Queue Implementation

Question: Implement message passing using message queues.

c

```

#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>
#include <unistd.h>

struct message {
    long msg_type;
    char msg_text[100];
};

int main() {
    key_t key = ftok("progfile", 65);
    int msgid = msgget(key, 0666 | IPC_CREAT);
    struct message msg;

    pid_t pid = fork();

    if (pid == 0) {
        // Child - receiver
        msgrcv(msgid, &msg, sizeof(msg.msg_text), 1, 0);
        printf("Receiver: Message received: %s\n", msg.msg_text);
    } else {
        // Parent - sender
        msg.msg_type = 1;
        strcpy(msg.msg_text, "Hello from parent process!");
        msgsnd(msgid, &msg, sizeof(msg.msg_text), 0);
        printf("Sender: Message sent: %s\n", msg.msg_text);

        wait(NULL);
        msgctl(msgid, IPC_RMID, NULL); // Remove message queue
    }

    return 0;
}

```

9. Advanced Synchronization Problems

Problem 15: Readers-Writers Problem

Question: Implement readers-writers problem with writer preference.

c

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int data = 0, readers_count = 0;
sem_t read_count_access, service_queue, resource_access;

void* reader(void* arg) {
    int reader_id = *(int*)arg;

    while (1) {
        // Entry section for readers
        sem_wait(&service_queue);
        sem_wait(&read_count_access);

        readers_count++;
        if (readers_count == 1) {
            sem_wait(&resource_access); // First reader locks resource
        }

        sem_post(&service_queue);
        sem_post(&read_count_access);

        // Reading section
        printf("Reader %d is reading data: %d\n", reader_id, data);
        sleep(1);

        // Exit section for readers
        sem_wait(&read_count_access);
        readers_count--;
        if (readers_count == 0) {
            sem_post(&resource_access); // Last reader unlocks resource
        }
        sem_post(&read_count_access);

        sleep(2);
    }
}

void* writer(void* arg) {
    int writer_id = *(int*)arg;

    while (1) {
        // Entry section for writers
        sem_wait(&service_queue);
```



```

sem_wait(&resource_access);
sem_post(&service_queue);

// Writing section
data++;
printf("Writer %d is writing data: %d\n", writer_id, data);
sleep(2);

// Exit section for writers
sem_post(&resource_access);

sleep(3);
}
}

int main() {
    pthread_t readers[3], writers[2];
    int reader_ids[] = {1, 2, 3};
    int writer_ids[] = {1, 2};

    // Initialize semaphores
    sem_init(&read_count_access, 0, 1);
    sem_init(&service_queue, 0, 1);
    sem_init(&resource_access, 0, 1);

    // Create reader threads
    for (int i = 0; i < 3; i++) {
        pthread_create(&readers[i], NULL, reader, &reader_ids[i]);
    }

    // Create writer threads
    for (int i = 0; i < 2; i++) {
        pthread_create(&writers[i], NULL, writer, &writer_ids[i]);
    }

    // Wait for threads (in practice, you'd have a termination condition)
    for (int i = 0; i < 3; i++) {
        pthread_join(readers[i], NULL);
    }
    for (int i = 0; i < 2; i++) {
        pthread_join(writers[i], NULL);
    }

    return 0;
}

```

Problem 16: Dining Philosophers Problem

Question: Solve dining philosophers problem to avoid deadlock.

c

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5
#define LEFT (philosopher_id + N - 1) % N
#define RIGHT (philosopher_id + 1) % N
#define THINKING 0
#define HUNGRY 1
#define EATING 2

int state[N];
sem_t mutex;
sem_t philosophers[N];

void test(int philosopher_id) {
    if (state[philosopher_id] == HUNGRY &&
        state[LEFT] != EATING &&
        state[RIGHT] != EATING) {
        state[philosopher_id] = EATING;
        printf("Philosopher %d takes forks %d and %d\n", philosopher_id, LEFT, philosopher_id);
        printf("Philosopher %d is eating\n", philosopher_id);
        sem_post(&philosophers[philosopher_id]);
    }
}

void take_forks(int philosopher_id) {
    sem_wait(&mutex);
    state[philosopher_id] = HUNGRY;
    printf("Philosopher %d is hungry\n", philosopher_id);
    test(philosopher_id);
    sem_post(&mutex);
    sem_wait(&philosophers[philosopher_id]);
}

void put_forks(int philosopher_id) {
    sem_wait(&mutex);
    state[philosopher_id] = THINKING;
    printf("Philosopher %d puts down forks %d and %d\n", philosopher_id, LEFT, philosopher_id);
    printf("Philosopher %d is thinking\n", philosopher_id);
    test(LEFT);
    test(RIGHT);
    sem_post(&mutex);
}
```

```

void* philosopher(void* arg) {
    int philosopher_id = *(int*)arg;

    while (1) {
        sleep(1); // Think
        take_forks(philosopher_id);
        sleep(2); // Eat
        put_forks(philosopher_id);
    }
}

int main() {
    pthread_t threads[N];
    int ids[N] = {0, 1, 2, 3, 4};

    sem_init(&mutex, 0, 1);
    for (int i = 0; i < N; i++) {
        sem_init(&philosophers[i], 0, 0);
        state[i] = THINKING;
    }

    for (int i = 0; i < N; i++) {
        pthread_create(&threads[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(threads[i], NULL);
    }

    return 0;
}

```

10. Advanced Memory Management

Problem 17: LRU Page Replacement Algorithm

Question: Implement Least Recently Used (LRU) page replacement.

c

```
#include <stdio.h>
#include <stdbool.h>

typedef struct {
    int page_number;
    int last_used_time;
} Frame;

int findLRU(Frame frames[], int n, int current_time) {
    int lru_index = 0;
    int min_time = frames[0].last_used_time;

    for (int i = 1; i < n; i++) {
        if (frames[i].last_used_time < min_time) {
            min_time = frames[i].last_used_time;
            lru_index = i;
        }
    }
    return lru_index;
}

int findPage(Frame frames[], int n, int page) {
    for (int i = 0; i < n; i++) {
        if (frames[i].page_number == page) return i;
    }
    return -1;
}

void lruPageReplacement(int pages[], int n, int capacity) {
    Frame frames[capacity];
    int page_faults = 0;
    int current_time = 0;

    // Initialize frames
    for (int i = 0; i < capacity; i++) {
        frames[i].page_number = -1;
        frames[i].last_used_time = -1;
    }

    printf("Page Reference String: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", pages[i]);
    }
    printf("\n\n");

    for (int i = 0; i < n; i++) {
```

```

int page = pages[i];
current_time++;

int page_index = findPage(frames, capacity, page);

if (page_index != -1) {
    // Page hit
    frames[page_index].last_used_time = current_time;
    printf("Page %d -> Hit | Frames: ", page);
} else {
    // Page fault
    page_faults++;

    // Find empty frame or LRU frame
    int replace_index = -1;
    for (int j = 0; j < capacity; j++) {
        if (frames[j].page_number == -1) {
            replace_index = j;
            break;
        }
    }

    if (replace_index == -1) {
        replace_index = findLRU(frames, capacity, current_time);
    }

    frames[replace_index].page_number = page;
    frames[replace_index].last_used_time = current_time;

    printf("Page %d -> Page Fault | Frames: ", page);
}

for (int j = 0; j < capacity; j++) {
    if (frames[j].page_number == -1) printf("- ");
    else printf("%d ", frames[j].page_number);
}
printf("\n");
}

printf("\nTotal Page Faults: %d\n", page_faults);
printf("Page Fault Rate: %.2f%%\n", (float)page_faults/n * 100);
}

int main() {
    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
    int n = sizeof(pages) / sizeof(pages[0]);
    int capacity = 3;

```

```
lruPageReplacement(pages, n, capacity);  
return 0;  
}
```

Problem 18: Fixed Partition Memory Allocation

Question: Implement fixed partition memory allocation with first-fit, best-fit, and worst-fit.

c

```

#include <stdio.h>
#include <stdbool.h>

#define MAX_PARTITIONS 10
#define MAX_PROCESSES 10

typedef struct {
    int size;
    bool allocated;
    int process_id;
} Partition;

typedef struct {
    int id;
    int size;
    bool allocated;
    int partition_assigned;
} Process;

void initializePartitions(Partition partitions[], int partition_sizes[], int n) {
    for (int i = 0; i < n; i++) {
        partitions[i].size = partition_sizes[i];
        partitions[i].allocated = false;
        partitions[i].process_id = -1;
    }
}

void firstFit(Partition partitions[], int num_partitions, Process processes[], int num_processes) {
    printf("\n=== FIRST FIT ALLOCATION ===\n");

    for (int i = 0; i < num_processes; i++) {
        processes[i].allocated = false;
        processes[i].partition_assigned = -1;

        for (int j = 0; j < num_partitions; j++) {
            if (!partitions[j].allocated && partitions[j].size >= processes[i].size) {
                partitions[j].allocated = true;
                partitions[j].process_id = processes[i].id;
                processes[i].allocated = true;
                processes[i].partition_assigned = j;
                break;
            }
        }
    }
}

// Display results

```



```

printf("Process\tSize\tPartition\tPartition Size\tInternal Fragmentation\n");
for (int i = 0; i < num_processes; i++) {
    if (processes[i].allocated) {
        int partition = processes[i].partition_assigned;
        int internal_frag = partitions[partition].size - processes[i].size;
        printf("%d\t%d\t%d\t\t%d\t\t%d\n",
            processes[i].id, processes[i].size, partition,
            partitions[partition].size, internal_frag);
    } else {
        printf("%d\t%d\t\tNot Allocated\n", processes[i].id, processes[i].size);
    }
}
}

void bestFit(Partition partitions[], int num_partitions, Process processes[], int num_processes) {
    printf("\n=== BEST FIT ALLOCATION ===\n");

    // Reset partitions
    for (int i = 0; i < num_partitions; i++) {
        partitions[i].allocated = false;
        partitions[i].process_id = -1;
    }

    for (int i = 0; i < num_processes; i++) {
        processes[i].allocated = false;
        processes[i].partition_assigned = -1;

        int best_idx = -1;
        int min_waste = __INT_MAX__;

        for (int j = 0; j < num_partitions; j++) {
            if (!partitions[j].allocated && partitions[j].size >= processes[i].size) {
                int waste = partitions[j].size - processes[i].size;
                if (waste < min_waste) {
                    min_waste = waste;
                    best_idx = j;
                }
            }
        }

        if (best_idx != -1) {
            partitions[best_idx].allocated = true;
            partitions[best_idx].process_id = processes[i].id;
            processes[i].allocated = true;
            processes[i].partition_assigned = best_idx;
        }
    }
}

```

// Display results

```
printf("Process\tSize\tPartition\tPartition Size\tInternal Fragmentation\n");
for (int i = 0; i < num_processes; i++) {
    if (processes[i].allocated) {
        int partition = processes[i].partition_assigned;
        int internal_frag = partitions[partition].size - processes[i].size;
        printf("%d\t%d\t%d\t%d\t%d\n",
            processes[i].id, processes[i].size, partition,
            partitions[partition].size, internal_frag);
    } else {
        printf("%d\t%d\tNot Allocated\n", processes[i].id, processes[i].size);
    }
}

void worstFit(Partition partitions[], int num_partitions, Process processes[], int num_processes) {
    printf("\n=== WORST FIT ALLOCATION ===\n");

    // Reset partitions
    for (int i = 0; i < num_partitions; i++) {
        partitions[i].allocated = false;
        partitions[i].process_id = -1;
    }

    for (int i = 0; i < num_processes; i++) {
        processes[i].allocated = false;
        processes[i].partition_assigned = -1;

        int worst_idx = -1;
        int max_waste = -1;

        for (int j = 0; j < num_partitions; j++) {
            if (!partitions[j].allocated && partitions[j].size >= processes[i].size) {
                int waste = partitions[j].size - processes[i].size;
                if (waste > max_waste) {
                    max_waste = waste;
                    worst_idx = j;
                }
            }
        }

        if (worst_idx != -1) {
            partitions[worst_idx].allocated = true;
            partitions[worst_idx].process_id = processes[i].id;
            processes[i].allocated = true;
            processes[i].partition_assigned = worst_idx;
        }
    }
}
```

```

    }
}

// Display results
printf("Process\tSize\tPartition\tPartition Size\tInternal Fragmentation\n");
for (int i = 0; i < num_processes; i++) {
    if (processes[i].allocated) {
        int partition = processes[i].partition_assigned;
        int internal_frag = partitions[partition].size - processes[i].size;
        printf("%d\t%d\t%d\t\t%d\t\t%d\n",
            processes[i].id, processes[i].size, partition,
            partitions[partition].size, internal_frag);
    } else {
        printf("%d\t%d\t\tNot Allocated\n", processes[i].id, processes[i].size);
    }
}
}

int main() {
    int partition_sizes[] = {100, 500, 200, 300, 600};
    int num_partitions = 5;

    Process processes[] = {{1, 212}, {2, 417}, {3, 112}, {4, 426}};
    int num_processes = 4;

    Partition partitions[MAX_PARTITIONS];

    // Test First Fit
    initializePartitions(partitions, partition_sizes, num_partitions);
    firstFit(partitions, num_partitions, processes, num_processes);

    // Test Best Fit
    initializePartitions(partitions, partition_sizes, num_partitions);
    bestFit(partitions, num_partitions, processes, num_processes);

    // Test Worst Fit
    initializePartitions(partitions, partition_sizes, num_partitions);
    worstFit(partitions, num_partitions, processes, num_processes);

    return 0;
}

```

11. Advanced Bash Scripting

Problem 19: Advanced File Processing Script

Question: Write a script to analyze log files and generate reports.

bash

```
#!/bin/bash
```

```
# Advanced log analyzer script
```

```
log_file="/var/log/syslog"
```

```
output_dir="/log_analysis"
```

```
current_date=$(date +%Y-%m-%d)
```

```
# Create output directory
```

```
mkdir -p "$output_dir"
```

```
# Function to analyze log entries
```

```
analyze_logs() {
```

```
    echo "=== LOG ANALYSIS REPORT - $current_date ===" > "$output_dir/report.txt"
```

```
    echo "" >> "$output_dir/report.txt"
```

```
    # Count total lines
```

```
    total_lines=$(wc -l < "$log_file")
```

```
    echo "Total log entries: $total_lines" >> "$output_dir/report.txt"
```

```
    # Count error messages
```

```
    error_count=$(grep -c "ERROR\|Error\|error" "$log_file")
```

```
    echo "Error entries: $error_count" >> "$output_dir/report.txt"
```

```
    # Count warning messages
```

```
    warning_count=$(grep -c "WARNING\|Warning\|warning" "$log_file")
```

```
    echo "Warning entries: $warning_count" >> "$output_dir/report.txt"
```

```
    # Top 10 most frequent IP addresses
```

```
    echo "" >> "$output_dir/report.txt"
```

```
    echo "Top 10 IP addresses:" >> "$output_dir/report.txt"
```

```
    grep -o '[0-9]\{1,3\}\.[0-9]\{1,3\}\.[0-9]\{1,3\}\.[0-9]\{1,3\}' "$log_file" | \
```

```
    sort | uniq -c | sort -nr | head -10 >> "$output_dir/report.txt"
```

```
    # Hourly distribution
```

```
    echo "" >> "$output_dir/report.txt"
```

```
    echo "Hourly distribution:" >> "$output_dir/report.txt"
```

```
    awk '{print substr($3, 1, 2)}' "$log_file" | sort | uniq -c | sort -nr >> "$output_dir/report.txt"
```

```
}
```

```
# Function to backup old logs
```

```
backup_logs() {
```

```
    if [ -f "$log_file" ]; then
```

```
        backup_name="$output_dir/backup_$(date +%Y%m%d_%H%M%S).log"
```

```
        cp "$log_file" "$backup_name"
```

```
        echo "Log backed up to: $backup_name"
```

```
    fi
```

```

}

# Function to clean old analysis files
cleanup_old_reports() {
    find "$output_dir" -name "report_*.txt" -mtime +7 -delete
    echo "Cleaned up reports older than 7 days"
}

# Menu system
while true; do
    echo "===== LOG ANALYZER MENU ====="
    echo "1. Analyze current logs"
    echo "2. Backup logs"
    echo "3. Clean old reports"
    echo "4. View latest report"
    echo "5. Exit"
    echo "===== "
    read -p "Enter your choice (1-5): " choice

    case $choice in
        1)
            if [ -f "$log_file" ]; then
                echo "Analyzing logs..."
                analyze_logs
                echo "Analysis complete. Report saved to $output_dir/report.txt"
            else
                echo "Log file not found: $log_file"
            fi
            ;;
        2)
            backup_logs
            ;;
        3)
            cleanup_old_reports
            ;;
        4)
            if [ -f "$output_dir/report.txt" ]; then
                cat "$output_dir/report.txt"
            else
                echo "No report found. Run analysis first."
            fi
            ;;
        5)
            echo "Exiting..."
            exit 0
            ;;
        *)

```

```
    echo "Invalid choice. Please try again."
    ;;
esac
echo ""
done
```

Problem 20: System Monitoring Script

Question: Create a comprehensive system monitoring script.

```
bash
```

```
#!/bin/bash
```

```
# System monitoring script
```

```
REPORT_FILE="system_report_$(date +%Y%m%d_%H%M%S).txt"
```

```
THRESHOLD_CPU=80
```

```
THRESHOLD_MEMORY=90
```

```
THRESHOLD_DISK=85
```

```
# Function to get CPU usage
```

```
get_cpu_usage() {
```

```
    cpu_usage=$(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)
```

```
    echo "CPU Usage: ${cpu_usage}%"
```

```
    if (( $(echo "$cpu_usage > $THRESHOLD_CPU" | bc -l) )); then
```

```
        echo "WARNING: High CPU usage detected!"
```

```
    fi
```

```
}
```

```
# Function to get memory usage
```

```
get_memory_usage() {
```

```
    memory_info=$(free -m | grep '^Mem:')
```

```
    total_mem=$(echo $memory_info | awk '{print $2}')
```

```
    used_mem=$(echo $memory_info | awk '{print $3}')
```

```
    memory_percent=$((used_mem * 100 / total_mem))
```

```
    echo "Memory Usage: ${used_mem}MB / ${total_mem}MB (${memory_percent}%)"
```

```
    if [ $memory_percent -gt $THRESHOLD_MEMORY ]; then
```

```
        echo "WARNING: High memory usage detected!"
```

```
    fi
```

```
}
```

```
# Function to get disk usage
```

```
get_disk_usage() {
```

```
    echo "Disk Usage:"
```

```
    df -h | grep -E '^/dev/' | while read filesystem size used avail percent mount; do
```

```
        usage_num=$(echo $percent | cut -d'%' -f1)
```

```
        echo " $mount: $used/$size ($percent)"
```

```
        if [ $usage_num -gt $THRESHOLD_DISK ]; then
```

```
            echo " WARNING: High disk usage on $mount!"
```

```
        fi
```

```
    done
```

```
}
```

```
# Function to get network statistics
```



```

get_network_stats() {
    echo "Network Statistics:"
    for interface in $(ls /sys/class/net/ | grep -v lo); do
        if [ -f "/sys/class/net/$interface/statistics/rx_bytes" ]; then
            rx_bytes=$(cat /sys/class/net/$interface/statistics/rx_bytes)
            tx_bytes=$(cat /sys/class/net/$interface/statistics/tx_bytes)

            rx_mb=$((rx_bytes / 1024 / 1024))
            tx_mb=$((tx_bytes / 1024 / 1024))

            echo " $interface: RX ${rx_mb}MB, TX ${tx_mb}MB"
        fi
    done
}

# Function to get process information
get_process_info() {
    echo "Top 5 CPU consuming processes:"
    ps aux --sort=-%cpu | head -6 | tail -5

    echo ""
    echo "Top 5 Memory consuming processes:"
    ps aux --sort=-%mem | head -6 | tail -5
}

# Function to get system load
get_system_load() {
    load_avg=$(uptime | awk -F'load average:' '{print $2}')
    uptime_info=$(uptime | awk '{print $3,$4}' | sed 's/,//')

    echo "System uptime: $uptime_info"
    echo "Load average:$load_avg"
}

# Function to generate full report
generate_report() {
    echo "===== SYSTEM MONITORING REPORT =====" | tee $REPORT_FILE
    echo "Generated on: $(date)" | tee -a $REPORT_FILE
    echo "===== " | tee -a $REPORT_FILE
    echo "" | tee -a $REPORT_FILE

    get_system_load | tee -a $REPORT_FILE
    echo "" | tee -a $REPORT_FILE

    get_cpu_usage | tee -a $REPORT_FILE
    echo "" | tee -a $REPORT_FILE
}

```

```
get_memory_usage | tee -a $REPORT_FILE
echo "" | tee -a $REPORT_FILE

get_disk_usage | tee -a $REPORT_FILE
echo "" | tee -a $REPORT_FILE

get_network_stats | tee -a $REPORT_FILE
echo "" | tee -a $REPORT_FILE

get_process_info | tee -a $REPORT_FILE

echo "Report saved to: $REPORT_FILE"
}

# Function for real-time monitoring
real_time_monitor() {
    echo "Starting real-time monitoring (Press Ctrl+C to stop)..."
    while true; do
        clear
        echo "===== REAL-TIME SYSTEM MONITOR ====="
        echo "Last updated: $(date)"
        echo "===== "

        get_cpu_usage
        echo ""
        get_memory_usage
        echo ""
        get_system_load

        sleep 2
    done
}

# Main menu
case "$1" in
    "cpu")
        get_cpu_usage
        ;;
    "memory")
        get_memory_usage
        ;;
    "disk")
        get_disk_usage
        ;;
    "network")
        get_network_stats
        ;;
```

```

"processes")
    get_process_info
    ;;
"report")
    generate_report
    ;;
"monitor")
    real_time_monitor
    ;;
*)
    echo "Usage: $0 {cpu|memory|disk|network|processes|report|monitor}"
    echo ""
    echo "Options:"
    echo "  cpu      - Show CPU usage"
    echo "  memory   - Show memory usage"
    echo "  disk     - Show disk usage"
    echo "  network  - Show network statistics"
    echo "  processes - Show top processes"
    echo "  report   - Generate full report"
    echo "  monitor  - Start real-time monitoring"
    exit 1
    ;;
esac

```

ADVANCED VIVA QUESTIONS AND ANSWERS

12. Advanced Process Management

Q: What is the difference between preemptive and non-preemptive scheduling? A: In preemptive scheduling, OS can forcibly remove a process from CPU and allocate it to another process. In non-preemptive, a process runs until completion or voluntarily gives up CPU. Examples: Round Robin (preemptive), FCFS (non-preemptive).

Q: Explain context switching overhead. A: Context switching involves saving current process state (registers, PC, stack pointer) and loading new process state. Overhead includes time to save/restore state, TLB flush, cache misses. Frequent context switches reduce system throughput.

Q: What is process affinity in multicore systems? A: Process affinity is tendency of scheduler to keep process running on same CPU core to maintain cache locality. Soft affinity allows migration if necessary, hard affinity prevents migration.

Q: How does copy-on-write (COW) optimize fork()? A: After fork(), parent and child share same memory pages marked read-only. When either process writes, OS creates separate copy of that page. This delays memory copying until actually needed, saving time and memory.

13. Advanced Synchronization

Q: What is priority inversion and how to solve it? A: High-priority task blocked by low-priority task holding required resource, while medium-priority tasks run. Solutions: Priority inheritance (low-priority task temporarily gets high priority) or Priority ceiling (resource gets highest priority of tasks that can use it).

Q: What is the difference between spinlock and mutex? A: Spinlock keeps CPU busy-waiting in loop until lock available - good for short critical sections. Mutex puts thread to sleep until lock available - better for longer critical sections. Spinlocks waste CPU cycles but have lower latency.

Q: Explain the ABA problem in concurrent programming. A: Thread A reads value A, gets preempted. Thread B changes A to B then back to A. Thread A resumes, sees A and assumes nothing changed, but state may have changed. Solved using version numbers or hazard pointers.

14. Advanced Memory Management

Q: What is thrashing in virtual memory systems? A: When system spends more time paging than executing processes. Happens when working set size exceeds available physical memory. Pages are continuously swapped in/out. Solution: Reduce degree of multiprogramming or increase memory.

Q: Explain Translation Lookaside Buffer (TLB). A: Hardware cache storing recent virtual-to-physical address translations. Much faster than page table lookup. TLB miss requires page table walk. TLB hit provides direct address translation. Size typically 32-1024 entries.

Q: What is memory-mapped I/O vs port-mapped I/O? A: Memory-mapped: Device registers mapped to memory addresses, accessed using normal memory instructions. Port-mapped: Separate I/O address space, accessed using special I/O instructions (IN/OUT). Memory-mapped is more common in modern systems.

Q: Explain buddy system allocation algorithm. A: Memory divided into blocks of sizes 2^k . To allocate size S , find smallest $2^k \geq S$. If unavailable, split larger block recursively. To deallocate, merge with buddy if both free. Reduces external fragmentation but causes internal fragmentation.

15. Advanced File Systems

Q: What is journaling in file systems? A: Technique to maintain file system consistency. Before making changes, write description to journal. If crash occurs, replay journal on recovery. Types: Write-back (metadata+data), Write-through (metadata only), Write-around (data bypasses journal).

Q: Explain inode structure and direct/indirect blocks. A: Inode contains file metadata and block pointers. Direct blocks (typically 12) point directly to data blocks. Single indirect points to block containing pointers. Double indirect points to block containing single indirect pointers. Triple indirect adds another level.

Q: What is the difference between hard link and soft link? A: Hard link: Multiple directory entries pointing to same inode. Same file, different names. Can't span file systems. Soft link (symlink): File containing path to another file. Can span file systems, can point to non-existent files.

16. Advanced Deadlock Concepts

Q: What is the difference between deadlock prevention and avoidance? A: Prevention: Design system so deadlock is impossible (violate one of four conditions). Conservative approach. Avoidance: Allow deadlock conditions but avoid unsafe states using algorithms like Banker's. More flexible but requires future resource knowledge.

Q: Explain wait-die and wound-wait schemes. A: Both are timestamp-based deadlock prevention for databases. Wait-die: Older transaction waits, younger dies. Wound-wait: Older transaction wounds (aborts) younger, younger waits for older. Prevents circular waiting through age hierarchy.

Q: What is ostrich algorithm for deadlock? A: Ignore deadlock problem completely. Justified when deadlocks are rare and cost of prevention/avoidance exceeds cost of occasional reboot. Used in some PC systems. Named after ostrich burying head in sand.

17. Advanced I/O and Device Management

Q: Explain interrupt handling mechanism. A: 1) Device raises interrupt 2) CPU saves current state 3) CPU jumps to interrupt handler 4) Handler services interrupt 5) CPU restores saved state 6) Execution continues. Types: Maskable/non-maskable, vectored/pollled.

Q: What is DMA and its advantages? A: Direct Memory Access allows devices to transfer data directly to/from memory without CPU intervention. Advantages: Frees CPU for other tasks, faster data transfer, reduces CPU overhead. CPU only involved at start and completion.

Q: Explain polling vs interrupt-driven I/O. A: Polling: CPU repeatedly checks device status. Simple but wastes CPU cycles. Interrupt-driven: Device notifies CPU when ready. More efficient but adds interrupt overhead. Hybrid approach uses polling for high-frequency operations.

18. Advanced Networking and IPC

Q: What is the difference between message passing and shared memory? A: Message passing: Processes communicate by sending/receiving messages. No shared address space. Better for distributed systems. Shared memory: Processes share memory region. Faster but requires synchronization. Better for single-system communication.

Q: Explain socket programming concepts. A: Socket: Communication endpoint. Types: Stream (TCP - reliable, ordered), Datagram (UDP - unreliable, unordered). Operations: create, bind, listen, accept, connect, send/receive, close. Address families: AF_INET (IPv4), AF_UNIX (local).

Q: What is remote procedure call (RPC)? A: Mechanism allowing program to call procedure on remote machine as if local call. Client stub marshals parameters, sends to server. Server stub unmarshals, calls

local procedure, marshals results back. Provides location transparency.

19. Performance and Optimization

Q: What factors affect system call overhead? A: Context switch cost, parameter copying, privilege level change, cache effects, TLB flush, security checks. System calls are expensive - batch operations when possible, use memory mapping, reduce unnecessary calls.

Q: Explain cache coherence in multiprocessor systems. A: Problem: Multiple processors have copies of same memory location in their caches. Solution protocols: MSI (Modified, Shared, Invalid), MESI (adds Exclusive state), MOESI (adds Owned state). Ensures all processors see consistent view of memory.

Q: What is false sharing in multiprocessor systems? A: When different processors modify different variables in same cache line, causing unnecessary cache invalidations. Performance degradation despite no actual data sharing. Solution: Pad data structures to avoid sharing cache lines.

20. Security and Protection

Q: Explain different protection domains. A: User mode: Limited privileges, can't execute privileged instructions. Kernel mode: Full system access. Some systems have intermediate levels (rings in x86). Protection prevents user programs from damaging system or other programs.

Q: What is buffer overflow attack and how to prevent? A: Attack overwrites return address on stack to execute malicious code. Prevention: Stack canaries, non-executable stack (NX bit), address space layout randomization (ASLR), bounds checking, safe string functions.

Q: Explain access control mechanisms. A: Discretionary Access Control (DAC): Owner controls access. Mandatory Access Control (MAC): System enforces policy. Role-Based Access Control (RBAC): Permissions based on roles. Access Control Lists specify permissions per resource.

COMPLEX CODING PROBLEMS LIKELY IN EXAMS

Problem 21: Multi-level Feedback Queue Scheduling

Question: Implement a multi-level feedback queue scheduler.

c

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_QUEUES 3
#define MAX_PROCESSES 10

typedef struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int remaining_time;
    int current_queue;
    int completion_time;
    int turnaround_time;
    int waiting_time;
    bool completed;
    struct Process* next;
} Process;

typedef struct Queue {
    Process* front;
    Process* rear;
    int time_quantum;
} Queue;

// Queue operations
void enqueue(Queue* q, Process* p) {
    p->next = NULL;
    if (q->rear == NULL) {
        q->front = q->rear = p;
    } else {
        q->rear->next = p;
        q->rear = p;
    }
}

Process* dequeue(Queue* q) {
    if (q->front == NULL) return NULL;

    Process* p = q->front;
    q->front = q->front->next;
    if (q->front == NULL) q->rear = NULL;

    return p;
}
```

```

bool isEmpty(Queue* q) {
    return q->front == NULL;
}

void mlfq_scheduling(Process processes[], int n) {
    Queue queues[MAX_QUEUES];

    // Initialize queues with different time quantum
    queues[0].front = queues[0].rear = NULL;
    queues[0].time_quantum = 2; // Highest priority, shortest quantum

    queues[1].front = queues[1].rear = NULL;
    queues[1].time_quantum = 4; // Medium priority

    queues[2].front = queues[2].rear = NULL;
    queues[2].time_quantum = 8; // Lowest priority, longest quantum

    int current_time = 0;
    int completed_processes = 0;

    printf("Time\tProcess\tQueue\tRemaining\tAction\n");
    printf("----\t-----\t----\t-----\t-----\n");

    while (completed_processes < n) {
        // Check for arriving processes and add to highest priority queue
        for (int i = 0; i < n; i++) {
            if (processes[i].arrival_time == current_time) {
                processes[i].current_queue = 0; // Start at highest priority
                enqueue(&queues[0], &processes[i]);
                printf("%d\t%d\t%d\t%d\t\tArrived\n",
                    current_time, processes[i].pid, 0, processes[i].remaining_time);
            }
        }

        // Find highest priority non-empty queue
        Process* current_process = NULL;
        int selected_queue = -1;

        for (int i = 0; i < MAX_QUEUES; i++) {
            if (!isEmpty(&queues[i])) {
                current_process = dequeue(&queues[i]);
                selected_queue = i;
                break;
            }
        }
    }
}

```



```

if (current_process == NULL) {
    current_time++;
    continue;
}

// Execute process for time quantum or until completion
int execution_time = (current_process->remaining_time < queues[selected_queue].time_quantum) ?
    current_process->remaining_time : queues[selected_queue].time_quantum;

current_process->remaining_time -= execution_time;
current_time += execution_time;

printf("%d\t%d\t%d\t%d\t\tExecuting\n",
    current_time, current_process->pid, selected_queue, current_process->remaining_time);

if (current_process->remaining_time == 0) {
    // Process completed
    current_process->completion_time = current_time;
    current_process->turnaround_time = current_process->completion_time - current_process->arrival_time;
    current_process->waiting_time = current_process->turnaround_time - current_process->burst_time;
    current_process->completed = true;
    completed_processes++;

    printf("%d\t%d\t%d\t%d\t\tCompleted\n",
        current_time, current_process->pid, selected_queue, current_process->remaining_time);
} else {
    // Process not completed, move to lower priority queue
    if (selected_queue < MAX_QUEUES - 1) {
        current_process->current_queue = selected_queue + 1;
        enqueue(&queues[selected_queue + 1], current_process);
        printf("%d\t%d\t%d\t%d\t\tMoved to Q%d\n",
            current_time, current_process->pid, selected_queue,
            current_process->remaining_time, selected_queue + 1);
    } else {
        // Stay in lowest priority queue
        enqueue(&queues[selected_queue], current_process);
        printf("%d\t%d\t%d\t%d\t\tStayed in Q%d\n",
            current_time, current_process->pid, selected_queue,
            current_process->remaining_time, selected_queue);
    }
}
}
}

int main() {
    Process processes[] = {
        {1, 0, 8, 8, 0, 0, 0, 0, false, NULL},

```

```

    {2, 1, 4, 4, 0, 0, 0, 0, false, NULL},
    {3, 2, 9, 9, 0, 0, 0, 0, false, NULL},
    {4, 3, 5, 5, 0, 0, 0, 0, false, NULL}
};

int n = 4;

printf("Multi-Level Feedback Queue Scheduling\n");
printf("=====\n");
printf("Queue 0: Time Quantum = 2\n");
printf("Queue 1: Time Quantum = 4\n");
printf("Queue 2: Time Quantum = 8 (FCFS)\n\n");

mlfq_scheduling(processes, n);

printf("\nFinal Results:\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        processes[i].pid, processes[i].arrival_time, processes[i].burst_time,
        processes[i].completion_time, processes[i].turnaround_time, processes[i].waiting_time);
}

return 0;
}

```

Problem 22: Disk Scheduling Algorithms

Question: Implement FCFS, SSTF, SCAN, and C-SCAN disk scheduling algorithms.

c

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_REQUESTS 100

int compare(const void* a, const void* b) {
    return *(int*)a - *(int*)b;
}

void fcfs_disk_scheduling(int requests[], int n, int head) {
    printf("\n=== FCFS Disk Scheduling ===\n");
    int total_head_movement = 0;
    int current_head = head;

    printf("Order of service: %d", head);

    for (int i = 0; i < n; i++) {
        total_head_movement += abs(requests[i] - current_head);
        current_head = requests[i];
        printf(" -> %d", current_head);
    }

    printf("\nTotal head movement: %d\n", total_head_movement);
    printf("Average head movement: %.2f\n", (float)total_head_movement / n);
}

void sstf_disk_scheduling(int requests[], int n, int head) {
    printf("\n=== SSTF Disk Scheduling ===\n");
    bool visited[MAX_REQUESTS] = {false};
    int total_head_movement = 0;
    int current_head = head;

    printf("Order of service: %d", head);

    for (int i = 0; i < n; i++) {
        int min_distance = __INT_MAX__;
        int next_request = -1;

        // Find closest unvisited request
        for (int j = 0; j < n; j++) {
            if (!visited[j]) {
                int distance = abs(requests[j] - current_head);
                if (distance < min_distance) {
                    min_distance = distance;
                    next_request = j;
                }
            }
        }
    }
}

```

```

    }
}

visited[next_request] = true;
total_head_movement += min_distance;
current_head = requests[next_request];
printf("-> %d", current_head);
}

printf("\nTotal head movement: %d\n", total_head_movement);
printf("Average head movement: %.2f\n", (float)total_head_movement / n);
}

void scan_disk_scheduling(int requests[], int n, int head, int disk_size, bool direction) {
    printf("\n=== SCAN Disk Scheduling ===\n");
    printf("Direction: %s\n", direction ? "Right (increasing)" : "Left (decreasing)");

    int left[MAX_REQUESTS], right[MAX_REQUESTS];
    int left_count = 0, right_count = 0;

    // Separate requests into left and right of head
    for (int i = 0; i < n; i++) {
        if (requests[i] < head) {
            left[left_count++] = requests[i];
        } else {
            right[right_count++] = requests[i];
        }
    }

    // Sort both arrays
    qsort(left, left_count, sizeof(int), compare);
    qsort(right, right_count, sizeof(int), compare);

    int total_head_movement = 0;
    int current_head = head;
    printf("Order of service: %d", head);

    if (direction) { // Moving right
        // Service all requests to the right
        for (int i = 0; i < right_count; i++) {
            total_head_movement += abs(right[i] - current_head);
            current_head = right[i];
            printf("-> %d", current_head);
        }

        // Move to end of disk if there were requests to service
    }
}

```

```

if (right_count > 0) {
    total_head_movement += abs((disk_size - 1) - current_head);
    current_head = disk_size - 1;
    printf("-> %d (end)", current_head);
}

// Service requests to the left (in reverse order)
for (int i = left_count - 1; i >= 0; i--) {
    total_head_movement += abs(left[i] - current_head);
    current_head = left[i];
    printf("-> %d", current_head);
}
} else { // Moving left
    // Service all requests to the left (in reverse order)
    for (int i = left_count - 1; i >= 0; i--) {
        total_head_movement += abs(left[i] - current_head);
        current_head = left[i];
        printf("-> %d", current_head);
    }

    // Move to beginning of disk if there were requests to service
    if (left_count > 0) {
        total_head_movement += current_head;
        current_head = 0;
        printf("-> %d (start)", current_head);
    }

    // Service requests to the right
    for (int i = 0; i < right_count; i++) {
        total_head_movement += abs(right[i] - current_head);
        current_head = right[i];
        printf("-> %d", current_head);
    }
}

printf("\nTotal head movement: %d\n", total_head_movement);
printf("Average head movement: %.2f\n", (float)total_head_movement / n);
}

void cscan_disk_scheduling(int requests[], int n, int head, int disk_size) {
    printf("\n=== C-SCAN Disk Scheduling ===\n");

    int left[MAX_REQUESTS], right[MAX_REQUESTS];
    int left_count = 0, right_count = 0;

    // Separate requests into left and right of head
    for (int i = 0; i < n; i++) {

```

```

    if (requests[i] < head) {
        left[left_count++] = requests[i];
    } else {
        right[right_count++] = requests[i];
    }
}

// Sort both arrays
qsort(left, left_count, sizeof(int), compare);
qsort(right, right_count, sizeof(int), compare);

int total_head_movement = 0;
int current_head = head;
printf("Order of service: %d", head);

// Service all requests to the right
for (int i = 0; i < right_count; i++) {
    total_head_movement += abs(right[i] - current_head);
    current_head = right[i];
    printf("-> %d", current_head);
}

// Move to end of disk then to beginning
if (right_count > 0) {
    total_head_movement += abs((disk_size - 1) - current_head);
    total_head_movement += (disk_size - 1); // Jump to beginning
    current_head = 0;
    printf("-> %d (end) -> %d (start)", disk_size - 1, 0);
}

// Service requests from beginning (left side)
for (int i = 0; i < left_count; i++) {
    total_head_movement += abs(left[i] - current_head);
    current_head = left[i];
    printf("-> %d", current_head);
}

printf("\nTotal head movement: %d\n", total_head_movement);
printf("Average head movement: %.2f\n", (float)total_head_movement / n);
}

int main() {
    int requests[] = {176, 79, 34, 60, 92, 11, 41, 114};
    int n = 8;
    int head = 50;
    int disk_size = 200;

```

```

printf("Disk Scheduling Algorithms Comparison\n");
printf("=====\n");
printf("Requests: ");
for (int i = 0; i < n; i++) {
    printf("%d ", requests[i]);
}
printf("\nInitial head position: %d\n", head);
printf("Disk size: 0 to %d\n", disk_size - 1);

// Test all algorithms
fcfs_disk_scheduling(requests, n, head);
sstf_disk_scheduling(requests, n, head);
scan_disk_scheduling(requests, n, head, disk_size, true); // Moving right
cscan_disk_scheduling(requests, n, head, disk_size);

return 0;
}

```

Problem 23: Optimal Page Replacement (Belady's Algorithm)

Question: Implement optimal page replacement algorithm and compare with FIFO and LRU.

c

```

#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

int findOptimalReplace(int pages[], int n, int frames[], int capacity, int current_index) {
    int farthest = current_index;
    int replace_index = -1;

    for (int i = 0; i < capacity; i++) {
        int j;
        for (j = current_index + 1; j < n; j++) {
            if (frames[i] == pages[j]) {
                if (j > farthest) {
                    farthest = j;
                    replace_index = i;
                }
                break;
            }
        }

        // If page never appears again, replace it
        if (j == n) {
            return i;
        }
    }

    // If all pages will be used, replace the one used farthest in future
    return (replace_index == -1) ? 0 : replace_index;
}

void optimalPageReplacement(int pages[], int n, int capacity) {
    int frames[capacity];
    int page_faults = 0;

    // Initialize frames
    for (int i = 0; i < capacity; i++) {
        frames[i] = -1;
    }

    printf("\n=== OPTIMAL Page Replacement ===\n");
    printf("Page\tFrames\t\tAction\n");
    printf("----\t-----\t\t-----\n");

    for (int i = 0; i < n; i++) {
        int page = pages[i];
        bool page_found = false;
    }

```



```

// Check if page is already in memory
for (int j = 0; j < capacity; j++) {
    if (frames[j] == page) {
        page_found = true;
        break;
    }
}

if (!page_found) {
    page_faults++;

    // Find empty frame
    int empty_frame = -1;
    for (int j = 0; j < capacity; j++) {
        if (frames[j] == -1) {
            empty_frame = j;
            break;
        }
    }

    if (empty_frame != -1) {
        // Use empty frame
        frames[empty_frame] = page;
        printf("%d\t", page);
        for (int j = 0; j < capacity; j++) {
            if (frames[j] == -1) printf("- ");
            else printf("%d ", frames[j]);
        }
        printf("\tPage Fault (Empty frame)\n");
    } else {
        // Replace using optimal strategy
        int replace_index = findOptimalReplace(pages, n, frames, capacity, i);
        int replaced_page = frames[replace_index];
        frames[replace_index] = page;

        printf("%d\t", page);
        for (int j = 0; j < capacity; j++) {
            printf("%d ", frames[j]);
        }
        printf("\tPage Fault (Replaced %d)\n", replaced_page);
    }
} else {
    printf("%d\t", page);
    for (int j = 0; j < capacity; j++) {
        printf("%d ", frames[j]);
    }
}

```

```

        printf("\tHit\n");
    }
}

printf("\nTotal Page Faults: %d\n", page_faults);
printf("Page Fault Rate: %.2f%%\n", (float)page_faults/n * 100);
}

void comparePageReplacementAlgorithms(int pages[], int n, int capacity) {
    printf("Page Replacement Algorithms Comparison\n");
    printf("=====\n");
    printf("Reference String: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", pages[i]);
    }
    printf("\nFrame Size: %d\n", capacity);

    // FIFO Implementation for comparison
    int fifo_frames[capacity];
    int fifo_faults = 0, front = 0;
    for (int i = 0; i < capacity; i++) fifo_frames[i] = -1;

    printf("\n=== FIFO Results ===\n");
    for (int i = 0; i < n; i++) {
        bool found = false;
        for (int j = 0; j < capacity; j++) {
            if (fifo_frames[j] == pages[i]) {
                found = true;
                break;
            }
        }
        if (!found) {
            fifo_frames[front] = pages[i];
            front = (front + 1) % capacity;
            fifo_faults++;
        }
    }
    printf("FIFO Page Faults: %d\n", fifo_faults);

    // Call optimal algorithm
    optimalPageReplacement(pages, n, capacity);

    printf("\n=== Comparison Summary ===\n");
    printf("FIFO Page Faults: %d\n", fifo_faults);
    printf("Optimal Page Faults: %d\n", (fifo_faults > page_faults ? page_faults : fifo_faults));
    printf("Improvement: %d fewer page faults\n", abs(fifo_faults - page_faults));
}

```

```
int main() {  
    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};  
    int n = sizeof(pages) / sizeof(pages[0]);  
    int capacity = 3;  
  
    comparePageReplacementAlgorithms(pages, n, capacity);  
  
    return 0;  
}
```

TRICKY VIVA QUESTIONS FOR HIGH MARKS

21. Advanced Concepts

Q: What happens if you call fork() in a multi-threaded process? A: Behavior is undefined in POSIX. Typically only calling thread is duplicated in child process, other threads don't exist in child. This can cause deadlocks if other threads held locks. Use fork-safe functions or exec immediately after fork.

Q: Why do some systems use segmented memory instead of paging? A: Segmentation provides logical protection boundaries matching program structure (code, data, stack segments). Each segment can have different access permissions. However, leads to external fragmentation. Modern systems use segmented paging - combining both approaches.

Q: What is memory barrier/fence and why needed? A: Prevents CPU/compiler from reordering memory operations across barrier. Needed in multi-processor systems where memory operations might complete out-of-order due to caching and optimization. Ensures memory ordering guarantees for correct synchronization.

Q: Explain the double-checked locking pattern and its problems. A: Pattern to reduce locking overhead in lazy initialization. Check condition, lock, check again, initialize. Problem: Without memory barriers, initialization might be partially visible to other threads due to instruction reordering. Needs proper memory ordering.

22. System Design Questions

Q: How would you design a process scheduler for a real-time system? A: Use priority-based preemptive scheduling. Hard real-time: Rate Monotonic (shorter period = higher priority) or Earliest Deadline First. Include priority inheritance to avoid priority inversion. Minimize interrupt latency. Provide timing guarantees.

Q: Design a file system that survives crashes. A: Use journaling: write-ahead log of all changes. Options: 1) Log metadata only (faster) 2) Log data+metadata (safer). On recovery, replay journal. Use checksums for corruption detection. Consider copy-on-write file systems like ZFS.

Q: How to implement virtual memory efficiently? A: Multi-level page tables to save space. TLB for fast address translation. Demand paging with intelligent replacement (LRU approximation). Use working set model. Memory mapping for files. Kernel virtual memory for internal structures.

23. Performance Analysis

Q: Your system has high context switch rate. How to investigate? A: 1) Use vmstat/sar to measure context switches 2) Check if caused by excessive thread creation 3) Look for lock contention causing frequent blocking 4) Check I/O patterns causing blocking 5) Consider thread pooling 6) Analyze scheduler statistics.

Q: Application performance degrades when more threads added. Why? A: Possible causes: 1) False sharing - threads access different data in same cache line 2) Lock contention - threads compete for shared resources 3) Memory bandwidth saturation 4) Context switching overhead 5) NUMA effects in multi-socket systems.

Q: How to detect memory leaks in long-running process? A: 1) Monitor process memory usage over time 2) Use tools like valgrind, AddressSanitizer 3) Check for growing heap with constant workload 4) Analyze allocation patterns 5) Use memory profilers 6) Check for unreleased resources (file descriptors, locks).

24. Security and Reliability

****Q: How**